

A decorative graphic on the left side of the slide consisting of two overlapping parallelograms. The front one is blue and the back one is a light greenish-blue. They are positioned diagonally, with the blue one partially covering the green one.

Predicting Credit Card Risk with Machine Learning Models

Jenish Bharucha and Romin Gandhi

Why Predict Credit Default?

Traditional rule-based systems are subjective and can't capture complex patterns



Goal: build a classifier that catches defaulters without flagging everyone

Only 1.7% of applicants default, extreme class imbalance makes this hard



Credit default costs financial institutions billions annually



Kaggle Credit Card Approval Prediction Dataset

- application_record.csv: 438,557 records, 18 attributes (demographics, income, employment, housing)
- credit_record.csv: 1,048,575 monthly payment status records
- One-to-many merge on applicant ID → aggregated to 36,457 usable records
- Target: default = 1 if ever 60+ days overdue (STATUS 2/3/4/5); 616 defaulters vs 35,841 non-defaulters

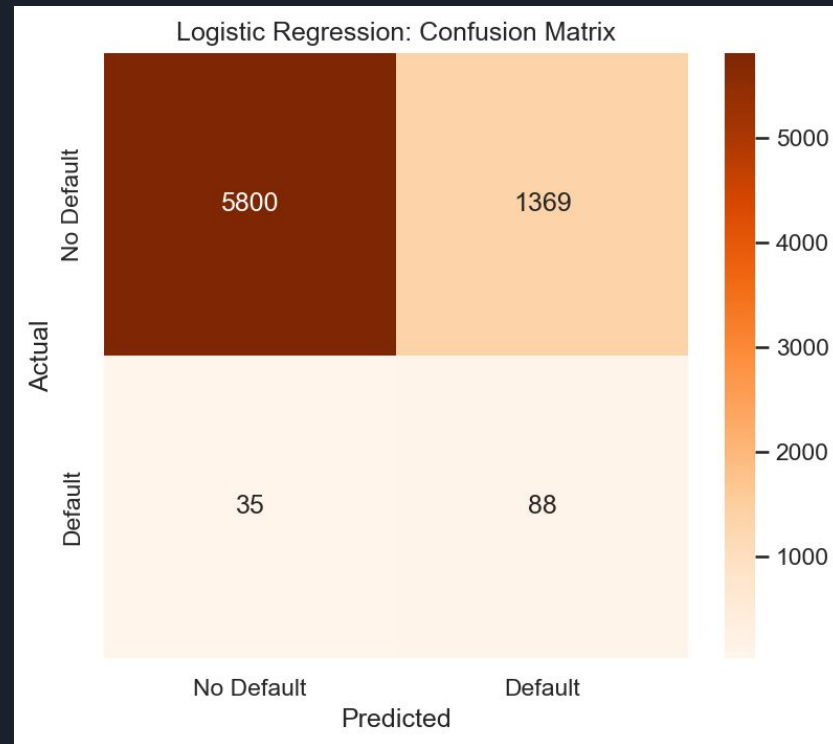


Preprocessing Pipeline

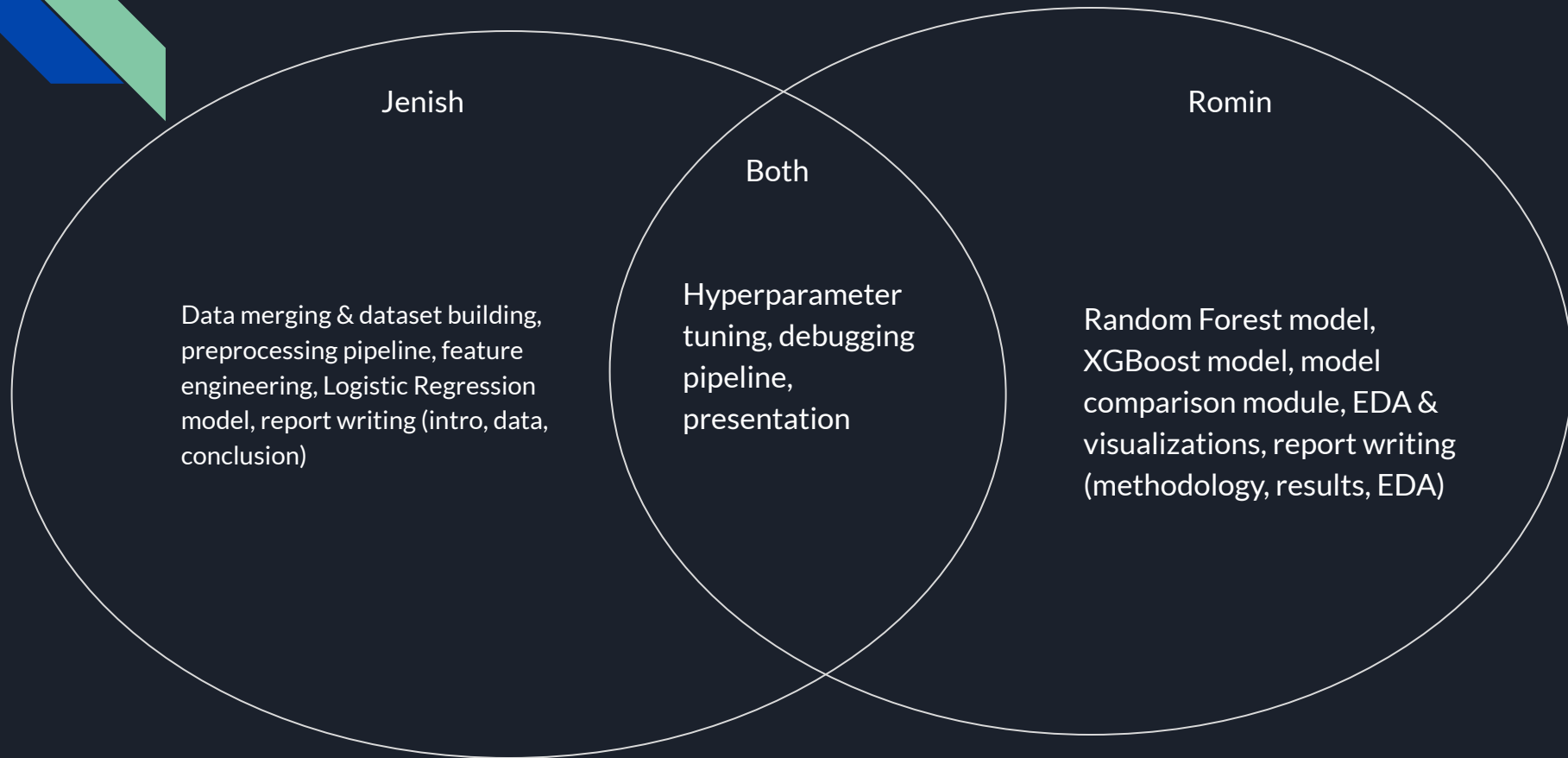
- Missing values: OCCUPATION_TYPE had 134K nulls — filled categorical with mode, numerical with median
- Outliers: IQR-based winsorization (capped at 1.5 x IQR boundaries)
- Encoding: Label encoding for binary columns, one-hot for multi-class (8 categorical features)
- Feature engineering: Created age_years, years_employed, income_per_member, income_per_child
- Scaling & split: StandardScaler on numericals, 80/20 stratified split → 29,165 train / 7,292 test

Logistic Regression: Baseline Model

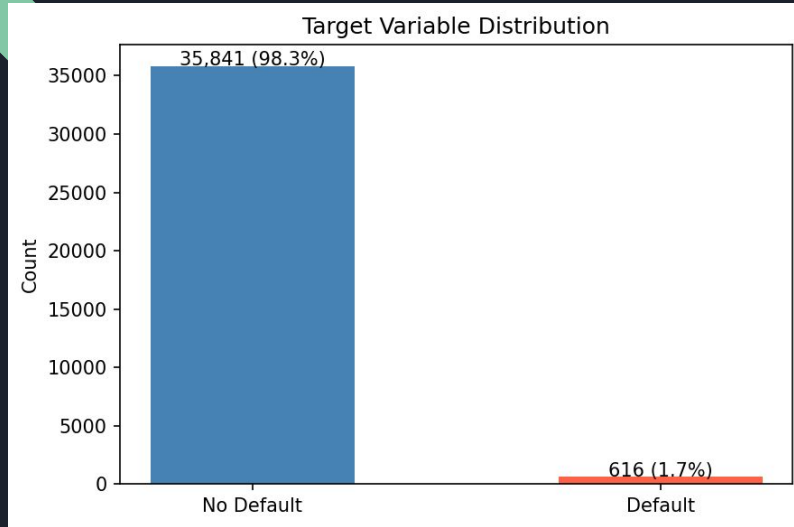
- SMOTE oversampling + GridSearchCV (5-fold CV)
- Searched: $C = [0.001, 0.01, 0.1, 1, 10, 100]$, penalty = L1/L2, solver = liblinear/saga
- Best params: $C=1$, L1 penalty, balanced class weight
- Results: 80.8% accuracy, 72% recall, 0.81 ROC AUC
- Trade-off: catches most defaulters but with many false positives (6% precision)



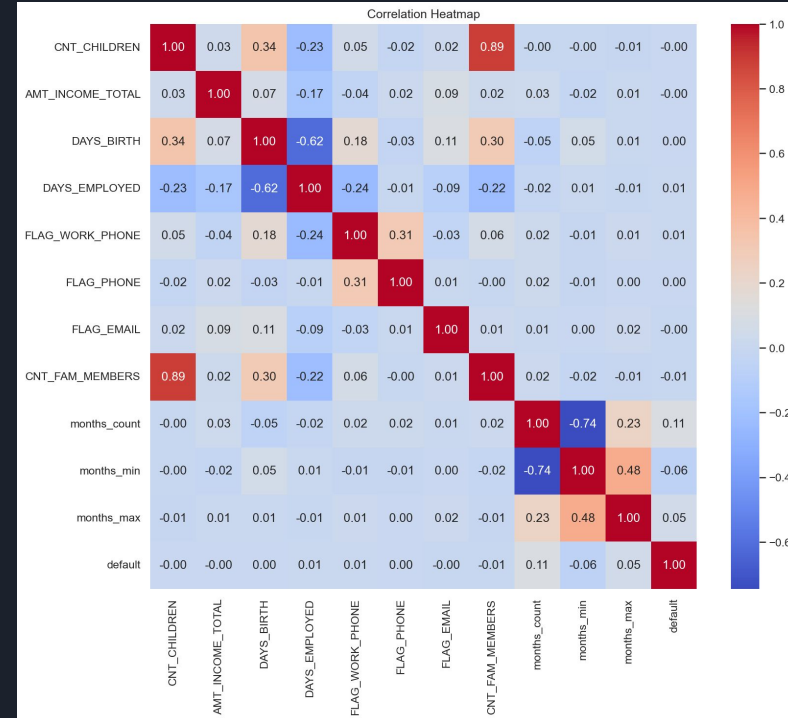
Work Distribution



Exploratory Data Analysis

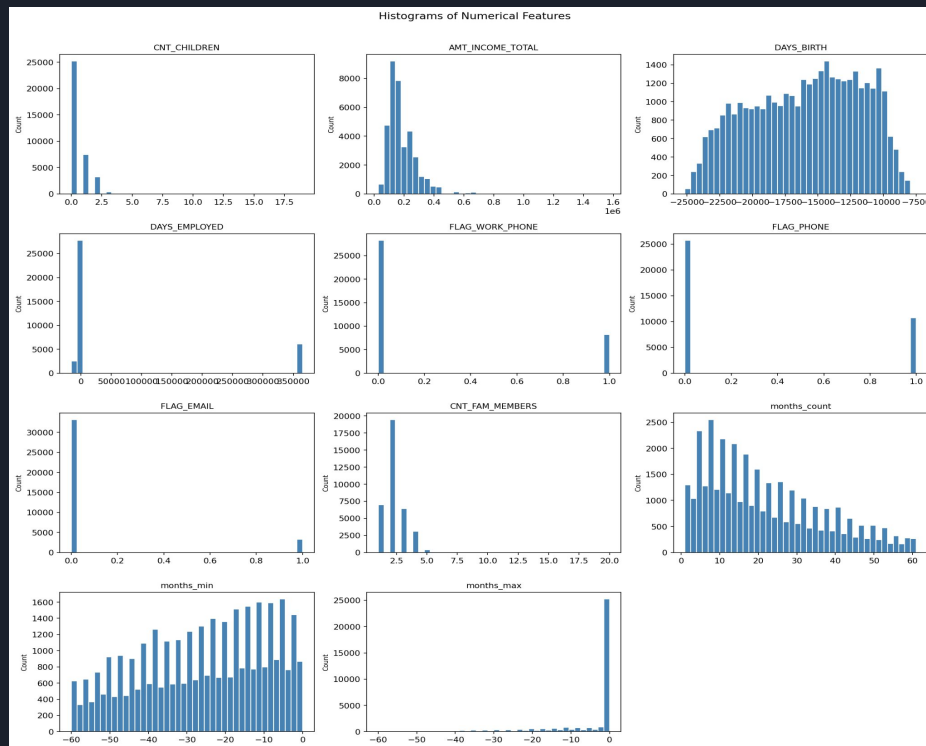


- Target is heavily imbalanced: 98.3% non-default vs 1.7% default (616 out of 36,457)
- OCCUPATION_TYPE has 134,203 missing values (~31% of records) – largest gap in the data
- Income distribution is right-skewed; most applicants earn under 380K annually
- Payment status "C" (paid off) dominates credit records – most applicants have healthy histories
- Correlation analysis: status_2 through status_5 counts are most predictive of default



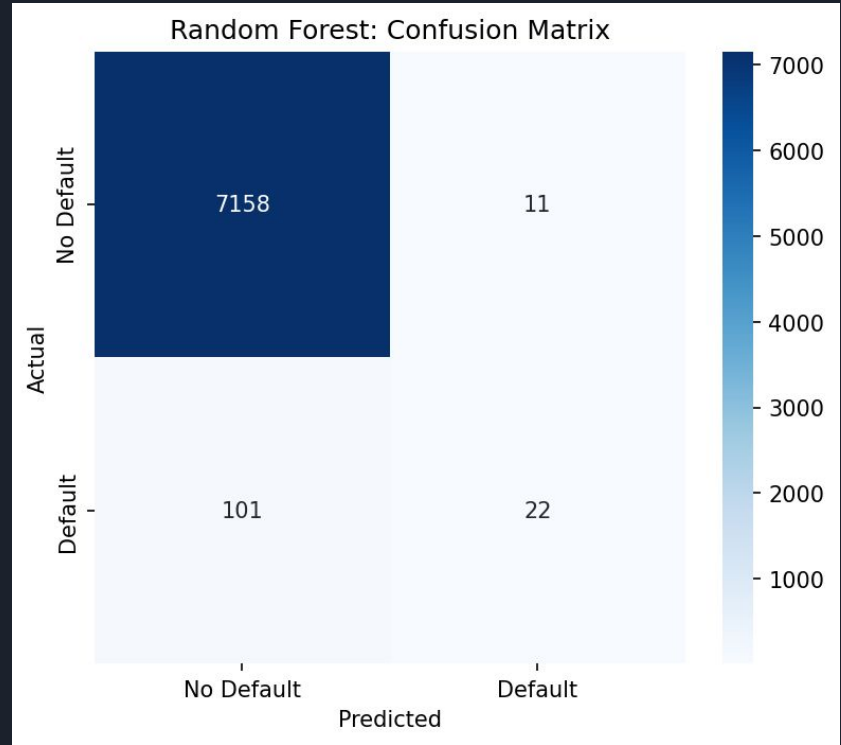
Feature Engineering

- Converted DAYS_BIRTH → age_years, DAYS_EMPLOYED → years_employed
- Created income_per_member (income / family size) and income_per_child
- Aggregated credit history: status frequency counts + months on record stats
- Removed low-variance features (threshold < 0.01) and highly correlated pairs ($r > 0.9$)
- Ranked features by mutual information — top predictors: status counts particularly status_0 and status_C and months_count

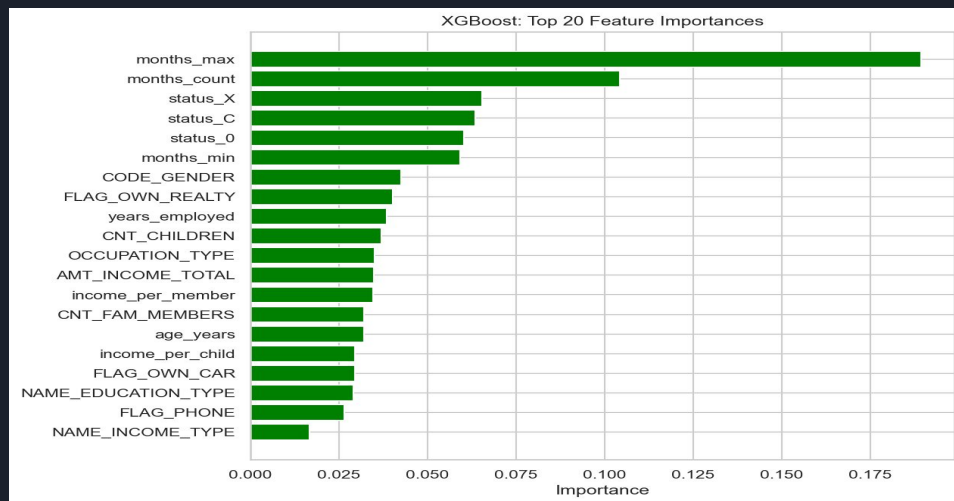


Random Forest

- SMOTE oversampling + balanced class weight in an imblearn pipeline
- RandomizedSearchCV (15 iterations, 5-fold CV, scoring = F1)
- Best params: 200 trees, max_depth=30, max_features=log2
- Results: 98.5% accuracy, 66.7% precision, 17.9% recall, 0.88 ROC AUC
- High precision but conservative — misses many defaulters to avoid false alarms



XGBoost



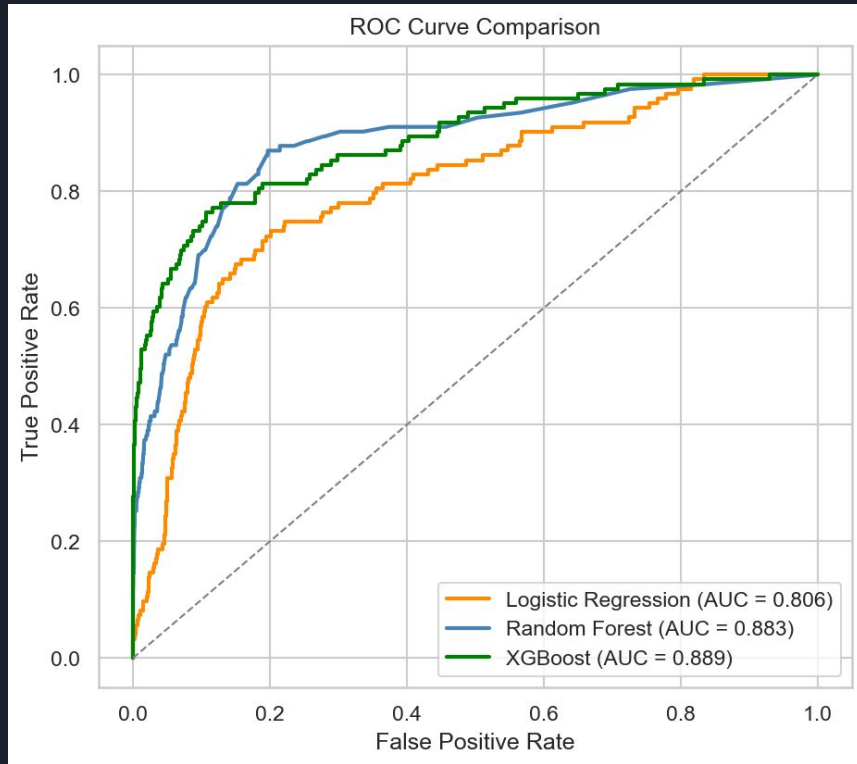
- Handles imbalance via scale_pos_weight (ratio ~58:1, auto-calculated from data)
- RandomizedSearchCV (20 iterations, 5-fold CV, scoring = F1)
- Best params: 300 trees, max_depth=5, learning_rate=0.2, subsample=1.0
- Results: 98.2% accuracy, 49.6% precision, 49.6% recall, 0.89 ROC AUC (highest)
- Best precision-recall balance among all three models



Model Comparison

Model	Accuracy	Precision	Recall	F1	ROC AUC
Logistic Regression	80.8%	6.0%	71.5%	0.11	0.81
Random Forest	98.5%	66.7%	17.9%	0.28	0.89
XGBoost	98.3%	49.6%	49.6%	0.50	0.89

ROC Comparison





Conclusion & Future Work

- XGBoost is the best model: highest ROC AUC (0.89), best F1 (0.50), 98.3% accuracy
- Ensemble methods significantly outperform logistic regression for imbalanced credit data
- Class imbalance handling (SMOTE, class weights, scale_pos_weight) is essential — not optional
- Future work: deep learning on sequential payment data, stacking all 3 models, fairness auditing across demographic groups